

CRYPTOGRAPHY

CTF Cheat Sheet | Ciphers, Encodings, Hash Cracking, RSA, Modern Crypto

STEP 0 — Identify What You Have First

Before anything else, figure out what the cipher/encoding is. Use this table:

It looks like...	It probably is...	Go to section
Letters only, looks shifted	Caesar / ROT cipher	Section 1
Letters only, random-looking	Vigenere / Substitution	Section 2 / 3
Only 0s and 1s	Binary encoding	Section 5
Hex chars (0-9, a-f only)	Hexadecimal	Section 5
Ends with = or ==	Base64	Section 5
Numbers with dots: 72.101.108	ASCII decimal	Section 5
Dots and dashes . - ...	Morse code	Section 4
Big N, e, c numbers	RSA	Section 7
32 hex chars	MD5 hash	Section 8
40 hex chars	SHA-1 hash	Section 8
64 hex chars	SHA-256 hash	Section 8
+, -, >, < symbols only	Brainfuck / esoteric	Section 9
Pigpen symbols / grids	Pigpen cipher	Section 4
bacon: AABBA AABAB	Baconian cipher	Section 4
Long number, p q given	RSA factor attack	Section 7
Random-looking with key	XOR cipher	Section 6
Looks like two ciphertexts	Polyalphabetic / OTP	Section 6

TIP: ALWAYS try CyberChef Magic mode first — gchq.github.io/CyberChef — paste ciphertext, click the wand icon. Solves 60% of beginner challenges automatically.

1. Caesar & ROT Ciphers

Caesar Cipher

Each letter is shifted by a fixed number. 25 possible keys.

```
# Brute force all 25 shifts (CyberChef -> ROT13 Brute Force)
# Or use dcode.fr/caesar-cipher -> Auto-solve

# Manual ROT13 (shift 13 — most common in CTFs)
A<->N B<->O C<->P D<->Q E<->R F<->S G<->T
H<->U I<->V J<->W K<->X L<->Y M<->Z

# Python: decode any Caesar shift
text = 'YRVVB'
for shift in range(26):
    print(shift, ''.join(chr((ord(c)-65-shift)%26+65) if c.isupper()
                        else chr((ord(c)-97-shift)%26+97) if c.islower()
                        else c for c in text))

# ROT47 (shifts all printable ASCII, not just letters)
# CyberChef -> ROT47
```

TIP: ROT13 is shift 13. Apply it twice to get back to original. Very common in CTFs.

ROT Quick Reference

ROT13 (shift 13)	A=N, B=O, C=P, D=Q, E=R, F=S, G=T, H=U, I=V, J=W, K=X, L=Y, M=Z
ROT47 (printable ASCII)	! -> P, A -> 2, Z -> z (shifts chars 33-126 by 47)
Online tool	rot13.com dcode.fr/caesar-cipher
CyberChef recipe	ROT13 -> tick 'Amount: 13' or brute-force all

2. Vigenere & Polyalphabetic Ciphers

Vigenere Cipher

Uses a keyword. Each letter shifted by corresponding key letter. Looks like random letters but has patterns.

```
# If KEY is given:
# CyberChef -> Vigenere Decode -> enter key
# dcode.fr/vigenere-cipher -> enter key

# If KEY is NOT given (most CTFs):
# dcode.fr/vigenere-cipher -> click 'Automatic Decryption'
# OR use: guballa.de/vigenere-solver
# OR use: www.dcode.fr/index-coincidence (find key length first)

# Python decode (if you have the key)
def vigenere_decrypt(ct, key):
    key = key.upper()
    result = ''
    ki = 0
    for c in ct.upper():
        if c.isalpha():
            result += chr((ord(c) - ord(key[ki % len(key)])) % 26 + 65)
            ki += 1
        else:
            result += c
    return result
print(vigenere_decrypt('CIPHERTEXT', 'KEY'))
```

TIP: If you see repeating patterns in the ciphertext, it's likely Vigenere. Key length = distance between repeated patterns.

Beaufort / Autokey Variants

```
# Beaufort: similar to Vigenere but different formula
# dcode.fr/beaufort-cipher

# Autokey: uses plaintext as part of key
# dcode.fr/autokey-cipher
```

3. Substitution Ciphers

Simple Substitution

Each letter maps to a different letter consistently throughout the message.

```
# Auto-solve (best tool for this):
# quipqiup.com -- paste ciphertext, click solve
# dcode.fr/monoalphabetic-substitution

# Frequency analysis hints:
```

```
# Most common letters in English: E T A O I N S H R
# Most common bigrams: TH HE IN ER AN
# Single letter words: A or I
# Most common 3-letter words: THE AND FOR

# If you see: 'Wkh iodj lv...' -> Caesar (common mistake to confuse with substitution)
| TIP: quipqiup.com is the best tool for substitution ciphers. It auto-solves using frequency analysis.
```

Atbash Cipher

```
# Mirror alphabet: A=Z, B=Y, C=X, D=W ...
# Self-inverse: apply twice to get original

A=Z B=Y C=X D=W E=V F=U G=T H=S I=R J=Q
K=P L=O M=N N=M O=L P=K Q=J R=I S=H T=G
U=F V=E W=D X=C Y=B Z=A

# CyberChef -> Atbash
# dcode.fr/atbash-cipher
```

4. Classical & Symbolic Ciphers

Morse Code

```
A .-   B -... C -. . D -.. E .   F ..-
G --.  H .... I ..  J .--- K -.- L .-..
M --   N -.   O --- P .--- Q --.- R .-
S ...  T -    U ..- V ...- W .-- X -.-
Y -.-- Z --..

1 .---- 2 ..--- 3 ...-- 4 ....- 5 .....
6 -.... 7 --... 8 ---.. 9 ----. 0 -----

# Decode online: morsecode.world | dcode.fr/morse-code
# CyberChef -> From Morse Code
```

Baconian Cipher

```
# Uses A and B (or 0 and 1): AAAAA=A, AAAAB=B, AAABA=C ...
# Each letter = 5 binary digits using A/B
A=AAAAA B=AAAAB C=AAABA D=AAABB E=AABAA
F=AABAB G=AABBA H=AABBB I/J=ABAAA K=ABAAB
L=ABABA M=ABABB N=ABBAA O=ABBAB P=ABBBA
Q=ABBBB R=BAAAA S=BAAAB T=BAABA U/V=BAABB
W=BABAA X=BABAB Y=BABBA Z=BABBB

# Decode: dcode.fr/baconian-cipher
```

Pigpen / Masonic Cipher

```
# Letters encoded as grid positions (symbols that look like squares/angles)
# If you see strange grid-like symbols -> Pigpen
# Decode: dcode.fr/pigpen-cipher
# Also try: dcode.fr/templars-cipher (cross-based variant)
```

Rail Fence Cipher

```
# Text written in zigzag across N rails, read row by row
# Example (3 rails): WEAREDISCOVEREDRUNATONCE
# Rail 1: W . . . E . . . I . . . V . . . D . . . E .
```

```
# Rail 2: . E . A . D . S . O . E . E . R . N . T . N . E
# Rail 3: . . R . . . C . . . R . . . U . . . A . . . C

# Decode: dcode.fr/rail-fence-cipher -> try different rail counts (2-10)
```

Playfair Cipher

```
# Digraph substitution – encrypts pairs of letters using a 5x5 grid
# Key = a keyword fills the grid, rest of alphabet follows
# Decode: dcode.fr/playfair-cipher -> enter key if known
```

5. Encodings (Not True Ciphers — Just Converted Format)

Base64

```
# Recognise: uses A-Z a-z 0-9 +/ and ends with = or ==
# Example: ZmxhZ3tleGFtcGxlfQ==

# Decode terminal:
echo 'ZmxhZ3tleGFtcGxlfQ==' | base64 -d

# Encode:
echo 'flag{example}' | base64

# Python:
import base64
base64.b64decode('ZmxhZ3tleGFtcGxlfQ==').decode()
base64.b64encode(b'flag').decode()

# Variants:
# Base32: uses A-Z 2-7, ends with = padding
echo 'MFRA====' | base32 -d
# Base58: used in Bitcoin, no 0 O l I characters
# Base85/Ascii85: ~5 chars per 4 bytes
# CyberChef handles all variants
```

Hexadecimal

```
# Recognise: only 0-9 and a-f, even number of chars
# Example: 666c61677b746573747d

# Terminal:
echo '666c61677b746573747d' | xxd -r -p
python3 -c "print(bytes.fromhex('666c61677b746573747d').decode())"

# CyberChef -> From Hex

# Encode to hex:
python3 -c "print('flag{test}'.encode().hex())"
xxd file.bin | head
```

Binary

```
# Recognise: only 0s and 1s, usually in groups of 8
# Example: 01100110 01101100 01100001 01100111

# Python:
binary = '01100110 01101100 01100001 01100111'
print(''.join(chr(int(b, 2)) for b in binary.split()))
```

```
# CyberChef -> From Binary

# Quick: each 8-bit group = 1 ASCII char
# 01000001 = 65 = A
```

ASCII Decimal

```
# Numbers representing ASCII values
# Example: 102 108 97 103 -> f l a g

# Python:
nums = [102, 108, 97, 103]
print(''.join(chr(n) for n in nums))

# ASCII table quick ref:
# 65-90 = A-Z    97-122 = a-z    48-57 = 0-9
# 32 = space    33 = !        123 = {    125 = }

# CyberChef -> From Charcode
```

URL Encoding

```
# %41 = A, %42 = B ... %7B = {, %7D = }
# Decode: CyberChef -> URL Decode
python3 -c "from urllib.parse import unquote; print(unquote('%66%6c%61%67'))"
```

HTML Entities

```
# &#102; = f    &#108; = l    &#97; = a    &#103; = g
# &lt; = <    &gt; = >    &amp; = &
# Decode: CyberChef -> HTML Entity Decode
```

Multiple Layers (Common in CTFs)

```
# CTFs often layer encodings: Base64 -> Hex -> ROT13 -> flag
# Use CyberChef 'Magic' mode to auto-detect layers
# Or manually: decode -> look at output -> decode again

# Common stacks:
Base64 of Base64
Hex of Base64
ROT13 then Base64
Binary then Base64
```

TIP: When CyberChef Magic doesn't work, manually try: Base64 decode -> check output -> Hex decode -> check -> ROT13. Repeat until flag appears.

6. XOR Cipher

XOR Basics

```
# XOR properties:
# A XOR A = 0    (XOR with itself = 0)
# A XOR 0 = A    (XOR with 0 = unchanged)
# A XOR B XOR B = A    (XOR is reversible)

# Single-byte XOR (key is one char - brute force all 256)
ct = bytes.fromhex('1a0ble0f1e') # your hex ciphertext
for key in range(256):
    pt = bytes([b ^ key for b in ct])
    if b'flag' in pt or b'CTF' in pt:
```

```

    print(key, pt)

# Multi-byte XOR (repeating key)
# Step 1: find key length using Index of Coincidence
# Step 2: split into columns, solve each as single-byte XOR
# Tool: dcode.fr/xor-cipher | CyberChef -> XOR Brute Force

# If you know part of plaintext (known-plaintext attack):
# key = plaintext XOR ciphertext
known = b'flag{'
ct_start = bytes([0x1a, 0x0b, 0x1e, 0x0f, 0x1e])
key = bytes([a ^ b for a, b in zip(known, ct_start)])
print(key)

# XOR two ciphertexts (OTP reuse):
# ct1 XOR ct2 = pt1 XOR pt2 (cancel out key)
ct1 = bytes.fromhex('CIPHERTEXT1')
ct2 = bytes.fromhex('CIPHERTEXT2')
xored = bytes([a ^ b for a, b in zip(ct1, ct2)])

```

TIP: If the flag format is known (e.g. 'flag{'), XOR the first bytes of ciphertext with 'flag{' to get the key, then decrypt the rest.

7. RSA Encryption

RSA Basics — Know This

n	modulus = $p * q$ (public)
e	public exponent (usually 65537)
d	private exponent (secret)
p, q	large prime factors of n (secret)
c	ciphertext = $m^e \bmod n$
m	plaintext (the flag) = $c^d \bmod n$
phi(n)	$(p-1) * (q-1)$
d formula	$d = \text{inverse}(e, \text{phi}(n))$

Attack 1 — Small n (Factor n)

```

# If n is small or has known factors:
# factordb.com -> paste n -> get p and q

# Python: factor small n
from sympy import factorint
print(factorint(n))

# Once you have p and q:
from Crypto.Util.number import inverse, long_to_bytes
p = ... # factor 1
q = ... # factor 2
phi = (p-1)*(q-1)
d = inverse(e, phi)
m = pow(c, d, n)
print(long_to_bytes(m))

```

Attack 2 — Small e (e=3, small message)

```

# If e=3 and message is small: c = m^3 (no modular reduction)

```

```
# Just take cube root of c

import gmpy2
m, exact = gmpy2.iroot(c, e) # e=3
from Crypto.Util.number import long_to_bytes
print(long_to_bytes(m))

# If e=3 and same message sent to 3 different n's: Hastad's broadcast attack
# Use CRT then cube root
```

Attack 3 — Common Factor (shared p or q)

```
# If two challenges share a prime factor in n:
from math import gcd
p = gcd(n1, n2) # finds shared prime
q1 = n1 // p
q2 = n2 // p
# Then decrypt normally
```

Attack 4 — Wiener's Attack (large d, small d)

```
# If d is small relative to n: Wiener's attack
# Tool: github.com/pablocelayes/rsa-wiener-attack
# Or: owiener Python library
import owiener
d = owiener.attack(e, n)
if d: print(pow(c, d, n))
```

Standard Decrypt (when p and q given)

```
from Crypto.Util.number import inverse, long_to_bytes

n = ... # given
e = ... # given (usually 65537)
c = ... # ciphertext
p = ... # given or found via factordb
q = n // p

phi = (p-1) * (q-1)
d = inverse(e, phi)
m = pow(c, d, n)
flag = long_to_bytes(m).decode()
print(flag)
```

TIP: Always check factordb.com first for the given n. Many CTF challenges use weak n values that are already factored in the database.

RSA Tools

factordb.com	Look up pre-factored n values
RsaCtfTool	<code>python3 RsaCtfTool.py --publickey key.pem --uncipherfile c.enc</code>
dcode.fr/rsa-cipher	Online RSA calculator
alpertron.com.ar/ECM	Factor large numbers
sagemath	<code>sage: factor(n) p,q = ...</code>
openssl	<code>openssl rsa -in key.pem -text -noout</code>

8. Hash Cracking

Identify Hash Type

32 hex chars	MD5
40 hex chars	SHA-1
56 hex chars	SHA-224
64 hex chars	SHA-256
96 hex chars	SHA-384
128 hex chars	SHA-512
\$2a\$ or \$2b\$...	bcrypt (very hard)
\$1\$...	MD5crypt
\$6\$...	SHA-512crypt
hash:salt format	Salted hash — need salt too

Crack a Hash

```
# 1. Try online first (rainbow tables):
crackstation.net      # Best — huge database
hashes.com            # Good alternative
md5decrypt.net        # MD5 specific
shal.gromweb.com      # SHA-1 specific

# 2. Hashcat (offline, GPU):
hashcat -a 0 -m 0      hash.txt rockyou.txt    # MD5
hashcat -a 0 -m 100    hash.txt rockyou.txt    # SHA-1
hashcat -a 0 -m 1400   hash.txt rockyou.txt    # SHA-256
hashcat -a 3 -m 0      hash.txt '?a?a?a?a?a?' # Brute 6 chars

# 3. John the Ripper:
john --wordlist=rockyou.txt --format=raw-md5 hash.txt
john --wordlist=rockyou.txt --format=raw-sha1 hash.txt

# 4. Python brute (tiny keyspace):
import hashlib
target = 'b14a7b8059d9c055954c92674ce60032'
wordlist = open('rockyou.txt', errors='ignore').read().splitlines()
for w in wordlist:
    if hashlib.md5(w.encode()).hexdigest() == target:
        print('Found:', w); break

# Hash identification tool:
hashid hash_value      # command line
hashes.com/detect      # online
```

TIP: Download rockyou.txt wordlist — it's the standard for CTF hash cracking. Most CTF hashes are common passwords.

Hash Length Extension Attack

```
# Vulnerable: MAC = hash(secret + message)
# Attack: append data without knowing secret
# Tool: hashpump
hashpump -s 'known_hash' -d 'known_data' -a ';admin=true' -k 16
# Also: hash_extender tool
```

9. Modern & Misc Crypto

AES Common Attacks

```
# AES-ECB (same plaintext block = same ciphertext block)
# Attack: rearrange or substitute ciphertext blocks
# Detect: encrypt 2 identical blocks -> identical output

# AES-CBC Bit Flip:
# Flip bit in ciphertext block N -> garbled block N, controlled flip in block N+1
# Formula:  $ct[i] \oplus old\_byte \oplus new\_byte$ 

# Padding Oracle:
# Server reveals if padding is valid -> decrypt byte by byte
# Tool: padbuster, python-paddingoracle

# IV = 0 or known IV:
#  $pt[0] \oplus IV = decrypted\_block[0]$  -> recover pt if IV known
```

Esoteric Languages

```
# Brainfuck: +, -, >, <, [, ], ., ,
# Run at: tio.run | copy.sh/brainfuck

# Malbolge: extremely complex, looks like symbols
# Run at: tio.run -> select Malbolge

# Ook!: Ook. Ook? Ook! (3 symbol combos)
# Decode: dcode.fr/ook-language

# Whitespace: hidden in spaces/tabs (looks blank)
# Decode: vii5ard.github.io/whitespace

# Piet: pixel art programs
# Run: bertnase.de/npiet/npiet-execute.php

# General esoteric runner: tio.run (supports 600+ languages)
```

Number Theory Tricks

```
# Modular arithmetic basics:
pow(base, exp, mod)      # Python: fast modular exponentiation

# Chinese Remainder Theorem (CRT):
from sympy.ntheory.modular import crt
r, m = crt([m1, m2, m3], [a1, a2, a3])  # solve  $x \equiv a_i \pmod{m_i}$ 

# Extended Euclidean (modular inverse):
from Crypto.Util.number import inverse
d = inverse(e, phi)      # modular inverse of e mod phi

# Discrete Log (small values):
#  $g^x \equiv h \pmod{p}$  -> find x
# Baby-step Giant-step for small p
from sympy.ntheory.residues import n_order
from sympy import discrete_log
x = discrete_log(p, h, g)
```

10. Complete Tools & Websites Reference

Tool / Site	Use for	Notes
gchq.github.io/CyberChef	Everything	USE MAGIC MODE FIRST — auto-detects encoding
dcode.fr	All classical ciphers	Has auto-solver for most ciphers
crackstation.net	Hash cracking	Largest free rainbow table database
factordb.com	RSA n factoring	Check if n is already factored
quipqiup.com	Substitution cipher	Best auto-solver for monoalphabetic
hashes.com	Hash ID + cracking	Identifies hash type automatically
jwt.io	JWT decode/forge	Decode and modify JWT tokens
guballa.de/vigenere-solver	Vigenere (no key)	Breaks Vigenere without knowing key
tio.run	Esoteric languages	Run Brainfuck, Malbolge, Ook etc.
rot13.com	ROT13/Caesar	Quick ROT decode
morsecode.world	Morse code	Decode/encode Morse
RsaCtfTool (GitHub)	RSA attacks	Automated RSA attack tool
hashcat	Offline hash crack	hashcat -a 0 -m 0 hash.txt rockyou.txt
john (JtR)	Offline hash crack	john --wordlist=rockyou.txt hash.txt
SageMath	Math/number theory	sage: factor(n), discrete_log etc.
alpertron.com.ar/ECM	Factor large n	ECM method for medium-sized n

11. Quick Decision Flowchart

Got a ciphertext? Follow this order:

STEP 1	Paste into CyberChef -> click Magic wand	<i>Solves ~60% of beginner challenges instantly</i>
STEP 2	Check: ends with =? -> Base64 decode	<i>base64decode.org or terminal: base64 -d</i>
STEP 3	Check: all hex chars (0-9,a-f)? -> Hex decode	<i>xxd -r -p or CyberChef From Hex</i>
STEP 4	Check: only 0s and 1s? -> Binary decode	<i>CyberChef From Binary</i>
STEP 5	Check: letters only, shifted? -> ROT/Caesar	<i>dcode.fr/caesar -> try all 25 shifts</i>
STEP 6	Check: big numbers N,e,c given? -> RSA	<i>factordb.com for n, then decrypt</i>
STEP 7	Check: 32/40/64 hex chars? -> Hash	<i>crackstation.net</i>
STEP 8	Still stuck? -> dcode.fr auto-solve	<i>Paste and let dcode identify cipher type</i>
STEP 9	Still stuck? -> Google exact ciphertext format	<i>Copy a distinctive fragment and Google it</i>